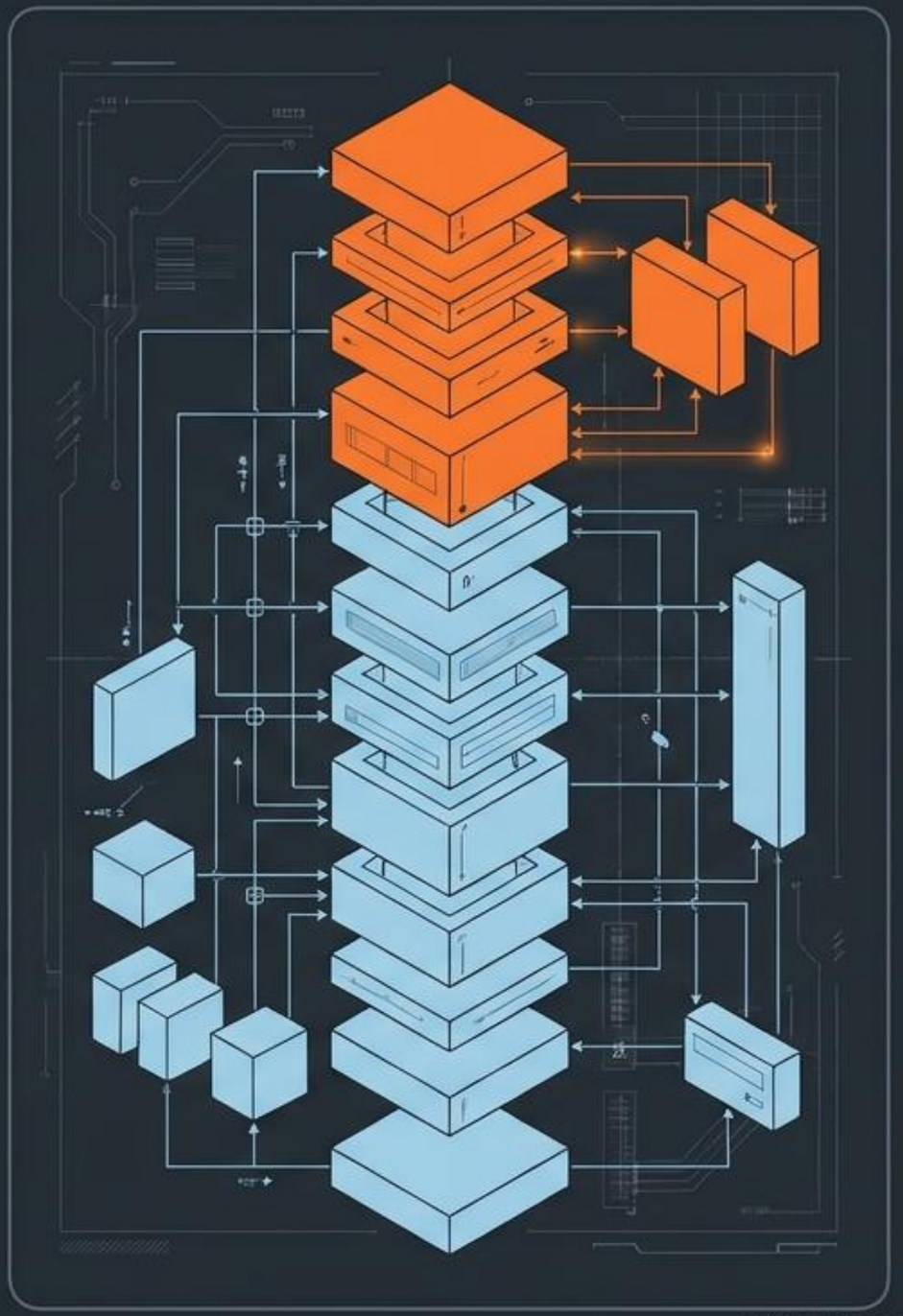
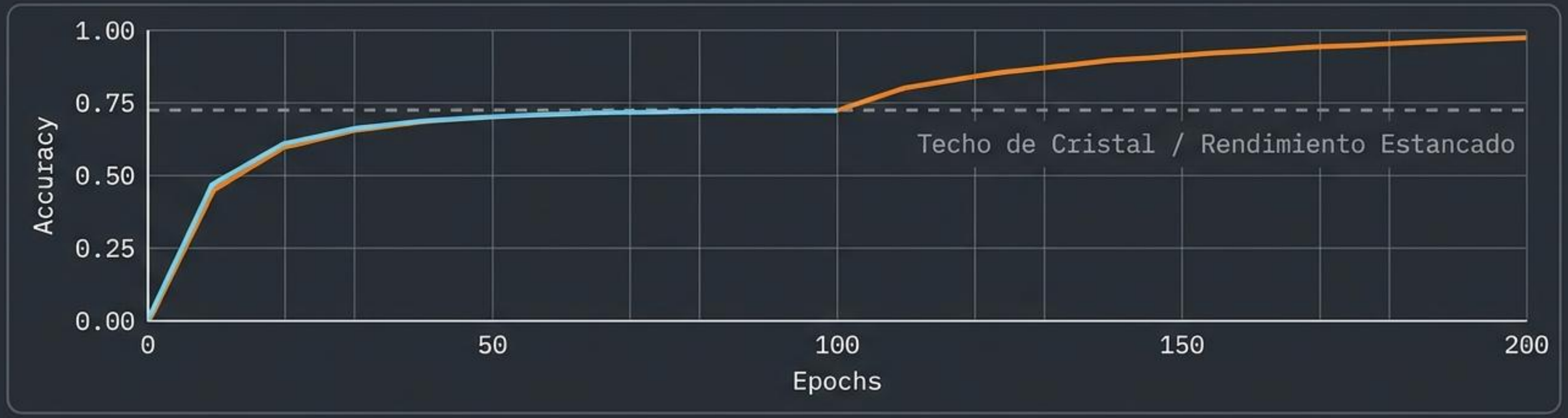
Sesión 09-B: Fine-Tuning y Estrategias Avanzadas

Llevando el Transfer Learning al nivel profesional



Curso IS056B — Ingeniería del Conocimiento
UNCP 2026-II | Unidad II
Docente: Msc. Jaime Huaytalla Pariona

EL TECHO DE CRISTAL DEL FEATURE EXTRACTION



El Límite del Feature Extraction

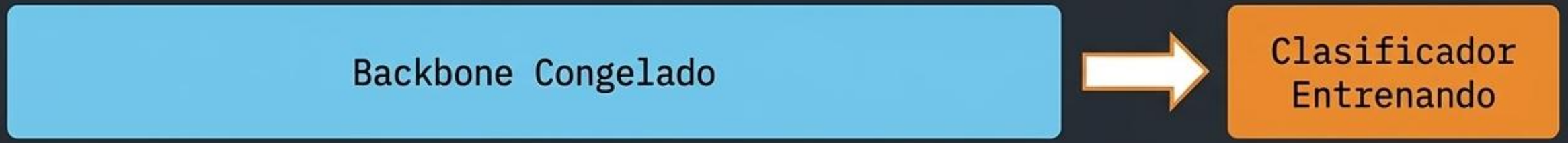
FE asume que los features de ImageNet son perfectos. Para dominios muy lejanos (ej. médico, satelital), las capas profundas codifican patrones subóptimos. El clasificador nuevo no puede compensarlo solo.

La Solución: Fine-Tuning

Abrir la caja negra.
Descongelamos estratégicamente parte del backbone y lo re-entrenamos con un learning rate bajo para adaptar sus features a nuestro dominio específico.

Fine-Tuning no es inmediato: El Protocolo de 2 Fases

Paso 1: Fase 1 (Warm-up) | 5-10 épocas



Propósito: El clasificador nuevo aprende a interpretar los features antes de modificarlos.

Paso 2: Fase 2 (Fine-Tuning) | 10-30 épocas



Propósito: Entrenar todo junto con Learning Rate microscópico.

¡Advertencia Crítica!
Si descongelas desde el inicio, el clasificador con pesos aleatorios enviará gradientes ruidosos como una onda expansiva, destruyendo el conocimiento pre-entrenado del backbone. La Fase 1 actúa como escudo protector.

Fase 1: Feature Extraction y estabilización

```
base_model = EfficientNetB0(weights='imagenet', include_top=False)
base_model.trainable = False

inputs = layers.Input(shape=(224, 224, 3))
x = preprocess_input(inputs)
x = base_model(x, training=False)
... # capas densas del head

model.compile(optimizer=Adam(learning_rate=1e-3), ...)
```

1. CONGELAR TODO

Bloquea la propagación de gradientes hacia el backbone. Mantiene el conocimiento intacto.

2. MODO INFERENCIA Y LR NORMAL

Se entrena solo el Head con un Learning Rate normal (1e-3). BN se mantiene bloqueado.

Fase 2: Descongelamiento Estratégico

```python
base_model.trainable = True

# Congelar excepto las últimas 30 capas
for layer in base_model.layers[:-30]:
    layer.trainable = False

# RECOMPILAR con LR mucho más bajo
model.compile(optimizer=Adam(learning_rate=1e-5), ..)

history2 = model.fit(...,
    initial_epoch=history1.epoch[-1] + 1,
    callbacks=[ModelCheckpoint, EarlyStopping, ReduceLROnPlateau])
```

1. LEARNING RATE MICROSCÓPICO

El LR debe ser 100x menor (ej. 1e-5) para no sobrescribir violentamente el conocimiento útil del modelo base.

2. CONTINUIDAD

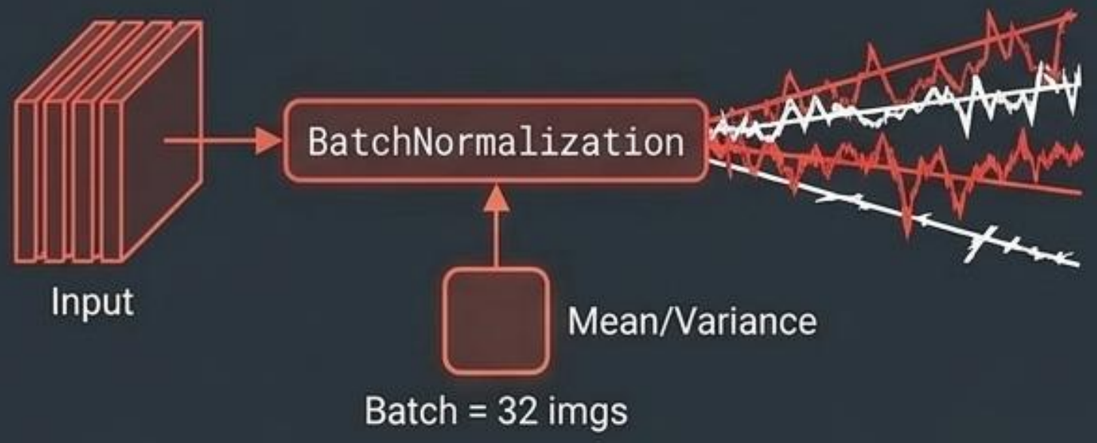
initial_epoch asegura que los gráficos de entrenamiento y logs continúen la numeración exacta de la Fase 1.



La trampa mortal de BatchNormalization

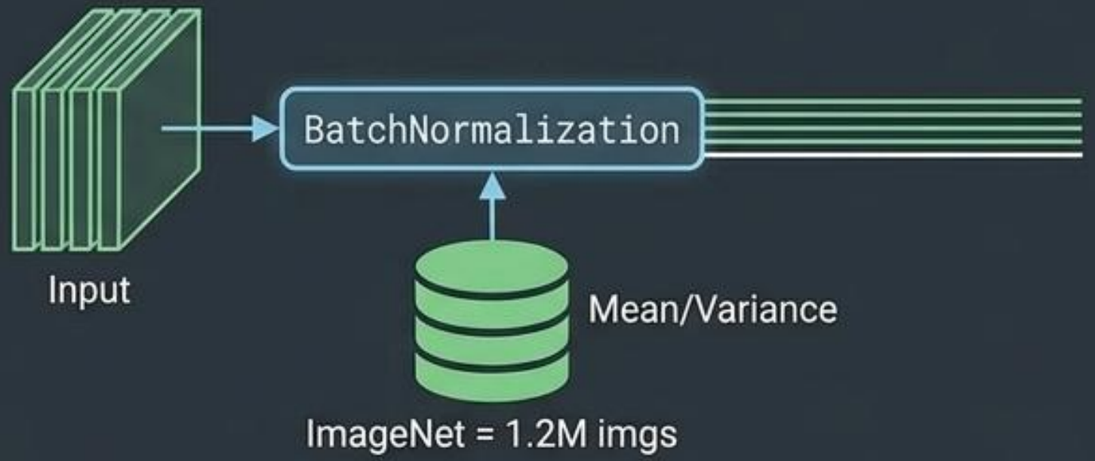
¿Por qué la mayoría de modelos divergen al hacer Fine-Tuning?
Porque omiten el comportamiento de las capas BN.

`training=True` (❌ Error Fatal)



Recalcula estadísticos sobre un batch minúsculo.
El ruido resultante destruye los features
pre-entrenados del backbone.

`training=False` (✅ Éxito Estructural)



Usa estadísticos congelados de la fuente original.
Garantiza estabilidad total de los tensores.

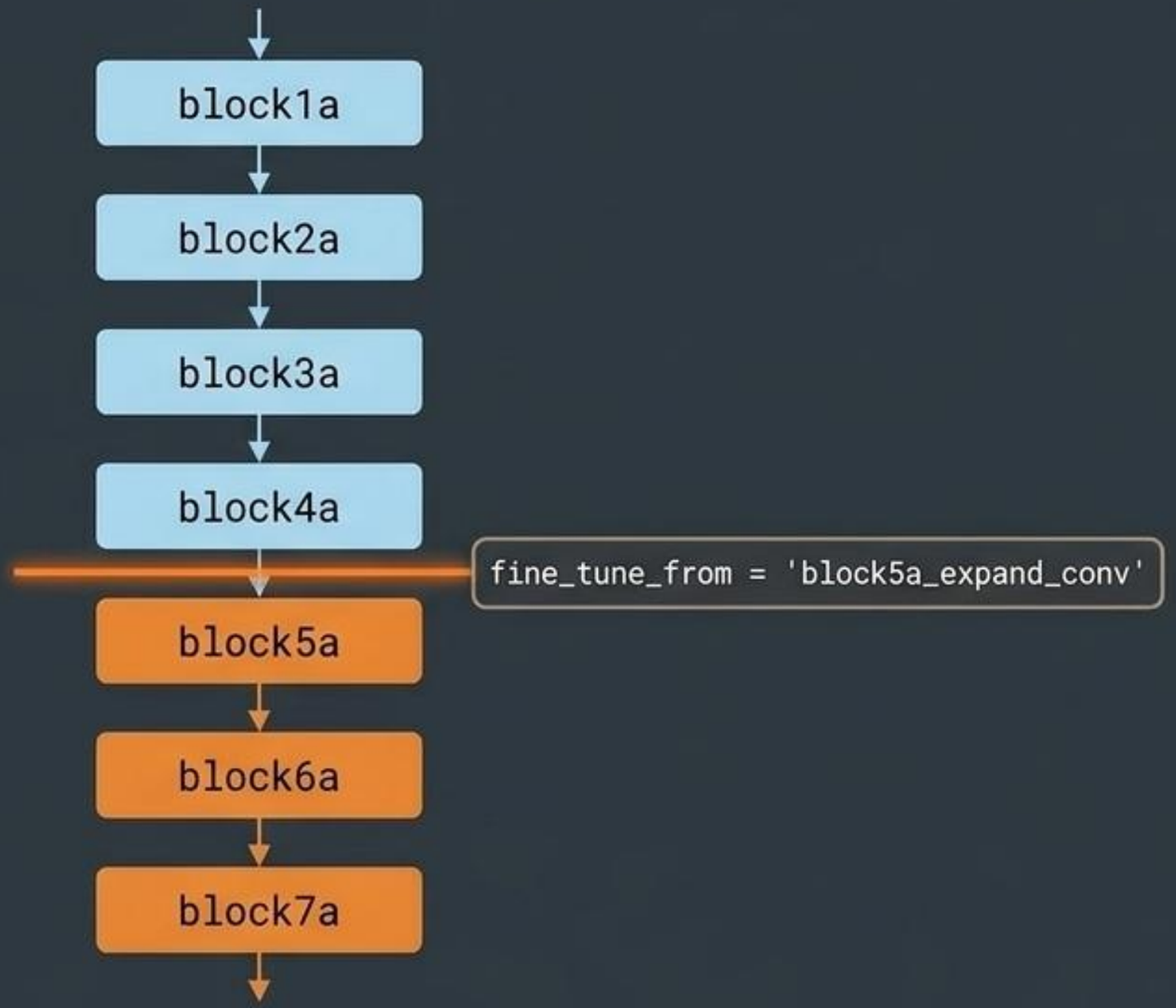
Regla de Oro: Incluso en la Fase 2 (capas descongeladas), la llamada `base_model(x, training=False)` NUNCA debe cambiar.

Matriz de Decisión: Profundidad del Fine-Tuning

	Similitud Alta (Ej. Fotos Naturales)	**Similitud Baja** (Ej. Médico, Satelital)
Dataset Pequeño (<1K imgs)	`Feature Extraction` puro.	`Feature Extraction` + Regularización fuerte (Mixup, Dropout).
Dataset Mediano (1K-10K imgs)	Descongelar últimas 20-30 capas.	Descongelar últimas 50-70 capas.
Dataset Grande (>10K imgs)	Descongelar todo (LR microscópico).	Fine-Tuning completo o Entrenar desde cero.

Precisión quirúrgica: Descongelando por bloques

Las arquitecturas modernas (ResNet, EfficientNet) operan en bloques lógicos secuenciales.
Romper un bloque descongelando capas sueltas genera **inestabilidad**.




```python
fine_tune_at = False
for layer in base_model.layers:
    # Buscar el punto exacto de corte arquitectónico
    if layer.name == 'block5a_expand_conv':
        fine_tune_at = True

    # Activar entrenamiento de aquí en adelante
    layer.trainable = fine_tune_at
```

Experimentación: Encontrando el Sweet Spot

No hay un número mágico de capas. El Fine-Tuning requiere búsqueda sistemática.
Más parámetros no siempre equivale a más accuracy.

Búsqueda Empírica (EfficientNetB0)

Capas Descongeladas	Parámetros Entrenables	Val Accuracy (5 ep)
10 capas	250,000	0.9410
✅ 30 capas	1,200,000	0.9650
50 capas	2,800,000	0.9520
❌ 100 capas	4,500,000	0.9200 (Sobreajuste)



Regularización Avanzada: Augmentation y Mixup

1. Augmentation Agresivo

```
RandomRotation(0.2)  // 20% vs 10%
RandomZoom(0.2)
RandomContrast(0.2)
```

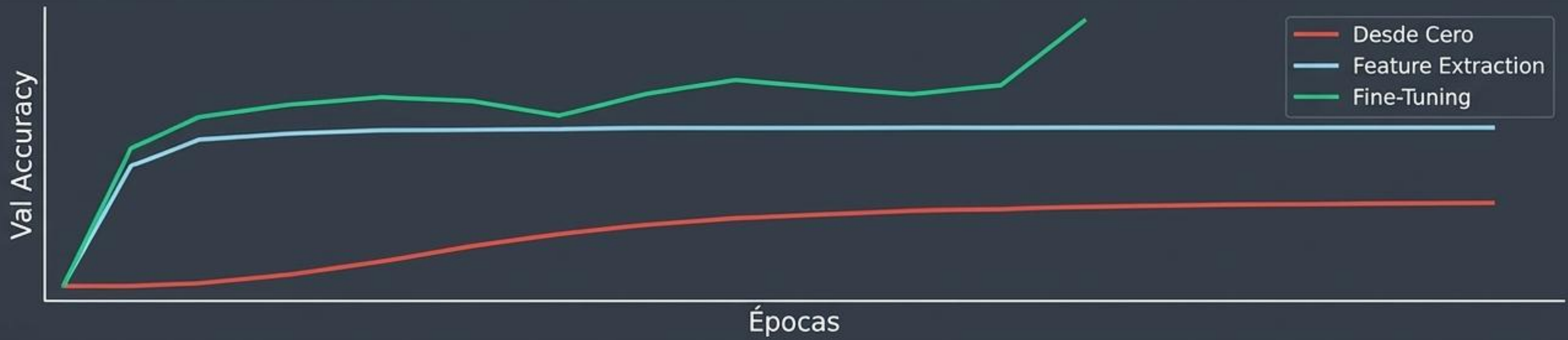
Al hacer Fine-Tuning, podemos ser agresivos. El backbone ya posee features robustos a la distorsión geométrica.

2. El Poder de Mixup



Obliga al modelo a aprender fronteras de decisión suaves, actuando como un regularizador extremo contra el sobreajuste.

Benchmarking: Desde cero vs. FE vs. Fine-Tuning



Resultados Típicos: PlantVillage | EfficientNetB0 | GPU T4

Estrategia	Accuracy Final	Épocas Totales	Tiempo Computacional
CNN Desde cero (S08)	85 - 90%	30-50	~2.0 horas
Feature Extraction	92 - 95%	5-10	~15 min
✅ Fine-Tuning (30 capas)	96 - 98%	25	~40 min

Métrica Clave: Fine-Tuning logra métricas de estado del arte consumiendo una fracción del costo computacional.

Supervivencia con micro-datasets (< 500 imágenes)

Regla de Oro: NUNCA usar Fine-Tuning.

El riesgo de sobreajuste es del 100%. Obligatorio utilizar Feature Extraction puro.

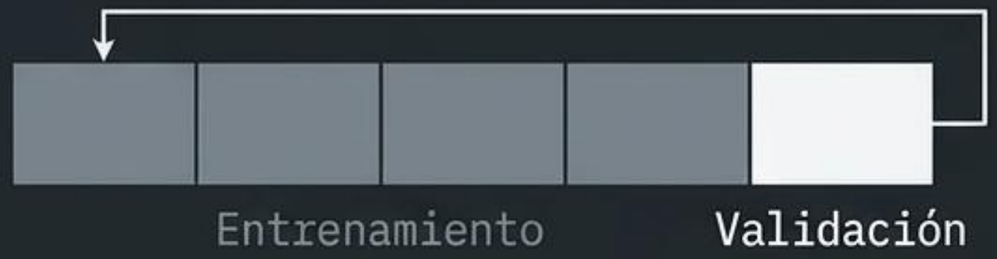
Estrategias de Rescate Estructurales

Usar **backbones** ultra-ligeros (MobileNetV2).

Dropout extremo (0.5) en el head.

Regularización L2 pesada en capas densas.

Evaluación Robusta (K-Fold)



Con <500 imágenes, el split clásico 80/20 produce métricas engañosas dictadas por la suerte. Promediar la validación con Stratified K-Fold es indispensable.

Del laboratorio a producción: Exportación

.keras
(Original)





.tflite
(Móvil/Web)

Script Inferencia: La Trampa Silenciosa

```
1  img_array = tf.expand_dims(img_array, 0)
2  # ¡CRUCIAL! Mismo preprocesamiento que el original
3  img_array = preprocess_input(img_array)
4  preds = model.predict(img_array)
```

CRUCIAL ADVERTENCIA

Un modelo exportado es basura en producción si la aplicación (web o móvil) no aplica exactamente el mismo escalado matemático de píxeles con el que fue entrenado el modelo base.

Checklist de Verificación de Transfer Learning

Evita errores silenciosos. Revisa estos puntos antes de dar por cerrado un proyecto de visión.

- [x] **include_top=False**: Se ejecutó para remover la arquitectura de clasificación original.
- [x] **training=False**: Todas las llamadas al backbone y capas BN están forzadas al modo inferencia.
- [x] **Fase 1 Completada**: El clasificador nuevo fue estabilizado (warm-up) antes de descongelar.
- [x] **Learning Rate Reducido**: En la Fase 2, el optimizador usa un LR 10x-100x más pequeño.
- [x] **Límites Arquitectónicos**: El descongelamiento se realizó cortando bloques completos, no capas al azar.
- [x] **Preprocesamiento Embebido**: La función de normalización matemática coincide entre entrenamiento y despliegue.

Misión Cumplida y Siguiente Desafío

Su Tarea

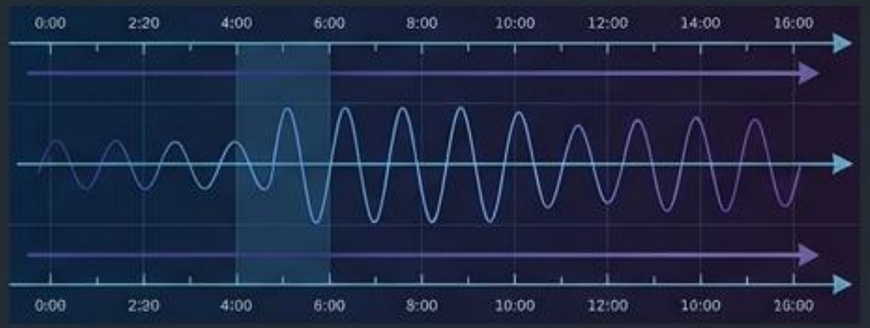


A partir de hoy, apliquen **Feature Extraction** y **Fine-Tuning** a sus propios datasets.

Las métricas que obtuvieron en su CNN de la S08 serán destruidas.

Reporten y analicen esta comparativa en su informe de investigación.

Preview Sesión 10: Series de Tiempo



Dejamos atrás las imágenes (datos espaciales). Entramos al mundo secuencial temporal.

La próxima semana trabajaremos con dependencias en el tiempo usando arquitecturas completamente nuevas: Redes Recurrentes (LSTM/GRU) y CNNs 1D.